Understanding Artificial Intelligence: From Mathematics to Applications

Lecture 3:
Regularization and the Bias-Variance tradeoff

Jean Feng

Logistics

- Homework 1 is due at midnight.
- Homework 2 is out.
- Start thinking about your project!

Lecture Outline

- **Nonlinear regression**
- Penalized linear regression
  - Ridge
  - Lasso
- Model Complexity

Quiz

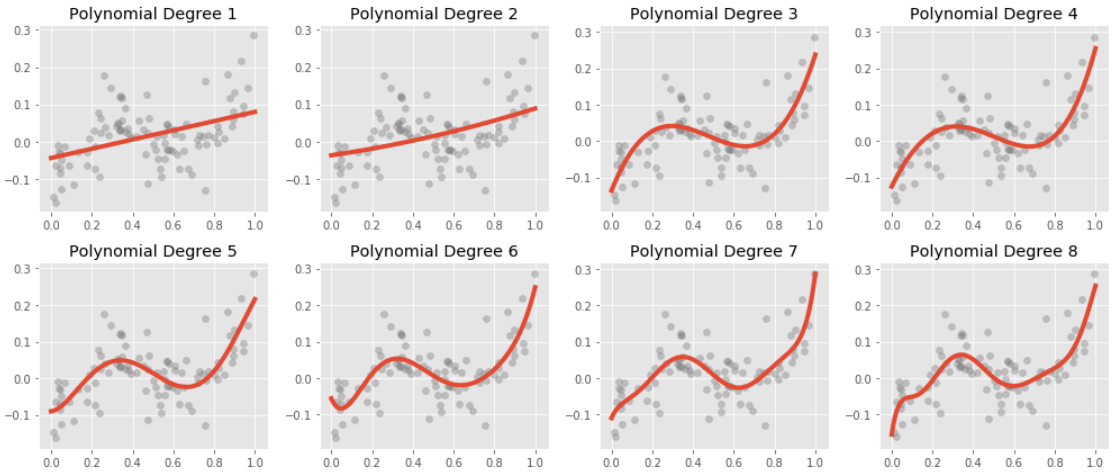
- **Q:** What happens if we don't think the outcome Y follows a linear relationship with respect to X_1, X_2, X_3? Can we use linear regression to fit a nonlinear model?
  - **A:** No.
  - **B:** Yes, we can fit a nonlinear model by running linear regression respect to the new set of features
    $X_1, \quad X_2, \quad X_3, \quad X_1 + X_2, \quad X_2 + X_3.$
  - **C:** Yes, we can fit a nonlinear model by running linear regression respect to the new set of features
    $X_1, \quad X_2, \quad X_3, \quad X_1 X_2, \quad X_2 X_3.$

Expanding the basis

- Change the **basis**, e.g. polynomial, spline, radial basis kernel, wavelets
  - Up to now, we've talked about the linear basis $\phi_j(x) = x_j$.
  - 2nd order polynomial basis: $\{x_1, x_1^2, x_2, x_2^2, x_1 x_2\}$
- **Sieve estimators** increase the size of the basis as data becomes more available. This lets us fit increasingly complex models, so it is a special type of nonparametric regression.

$$h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sum_{j=0}^{d} \theta_j \phi_j(\boldsymbol{x})$$

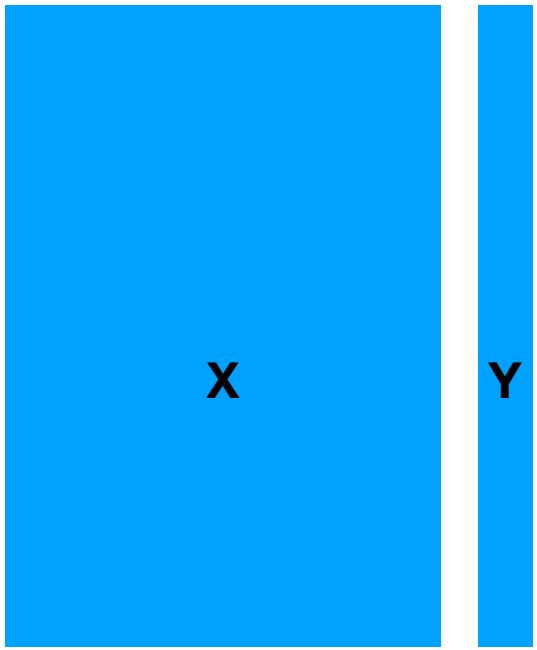
basis function

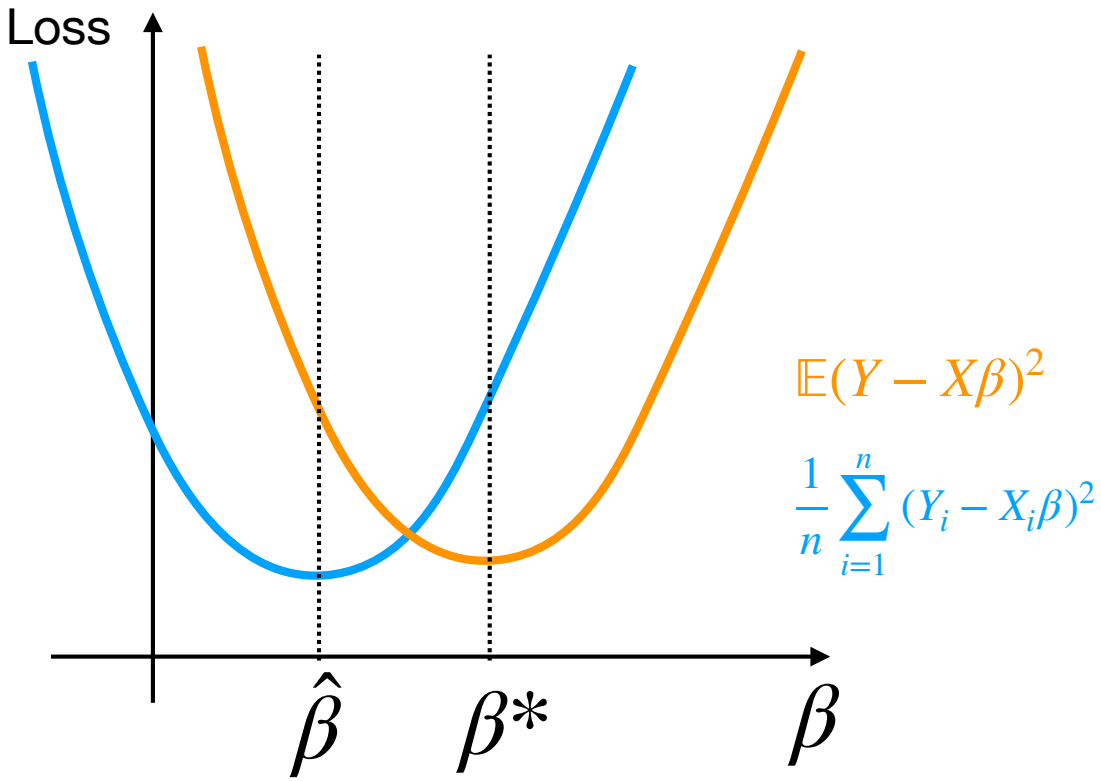


Lecture Outline

- Nonlinear regression
- **Penalized linear regression**
  - Ridge
  - Lasso
- Model Complexity

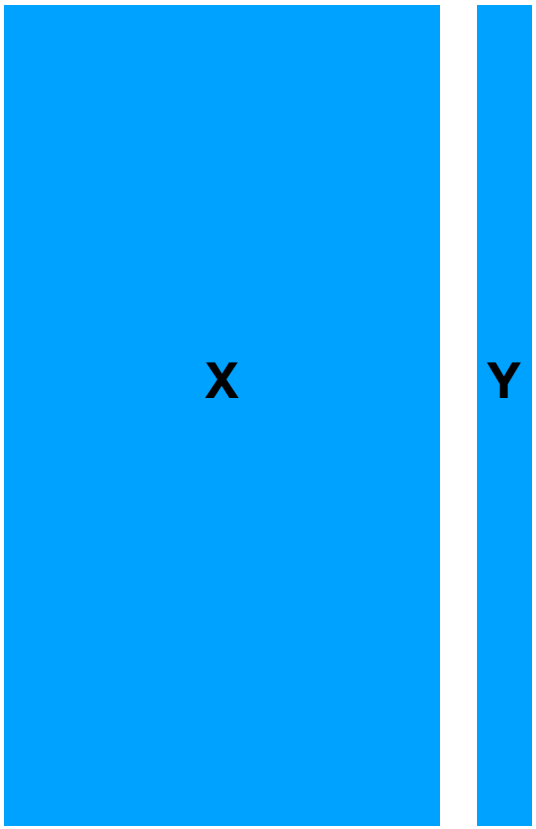
In low dimensional settings…



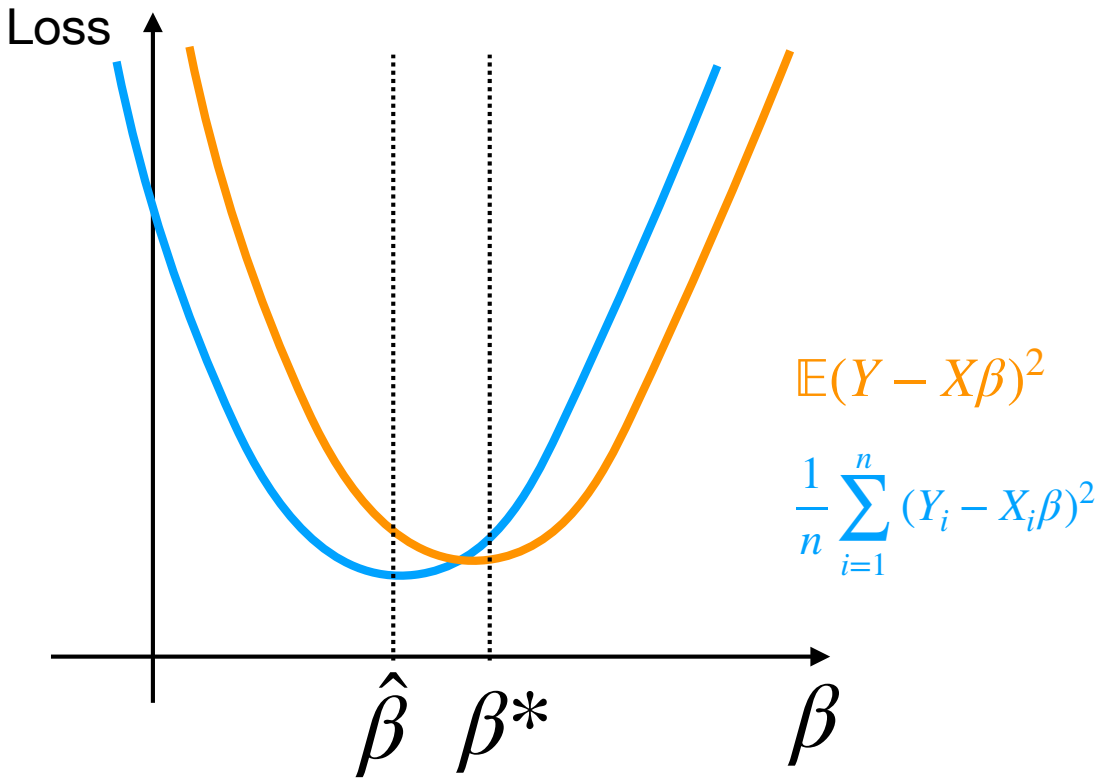


Low-dimensional

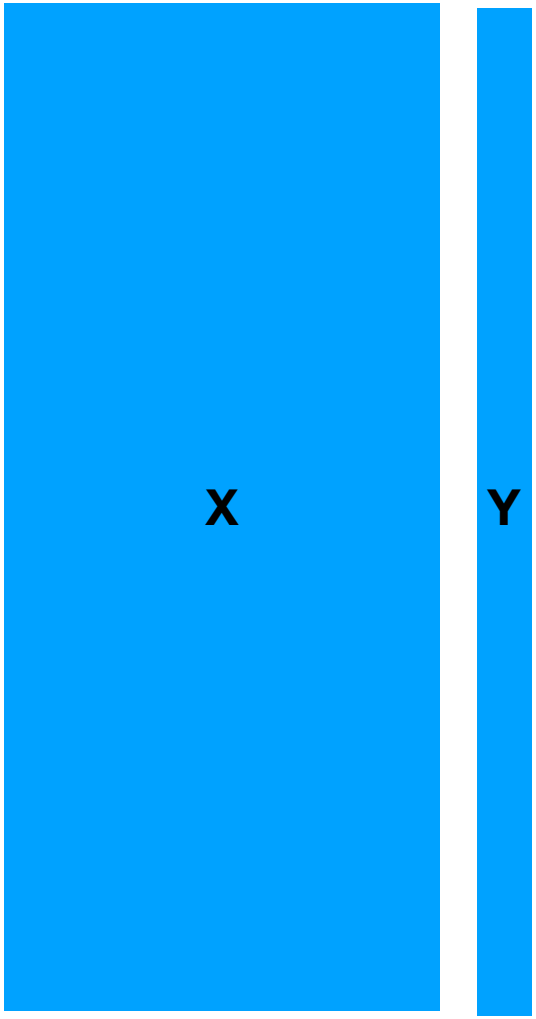
In low dimensional settings…



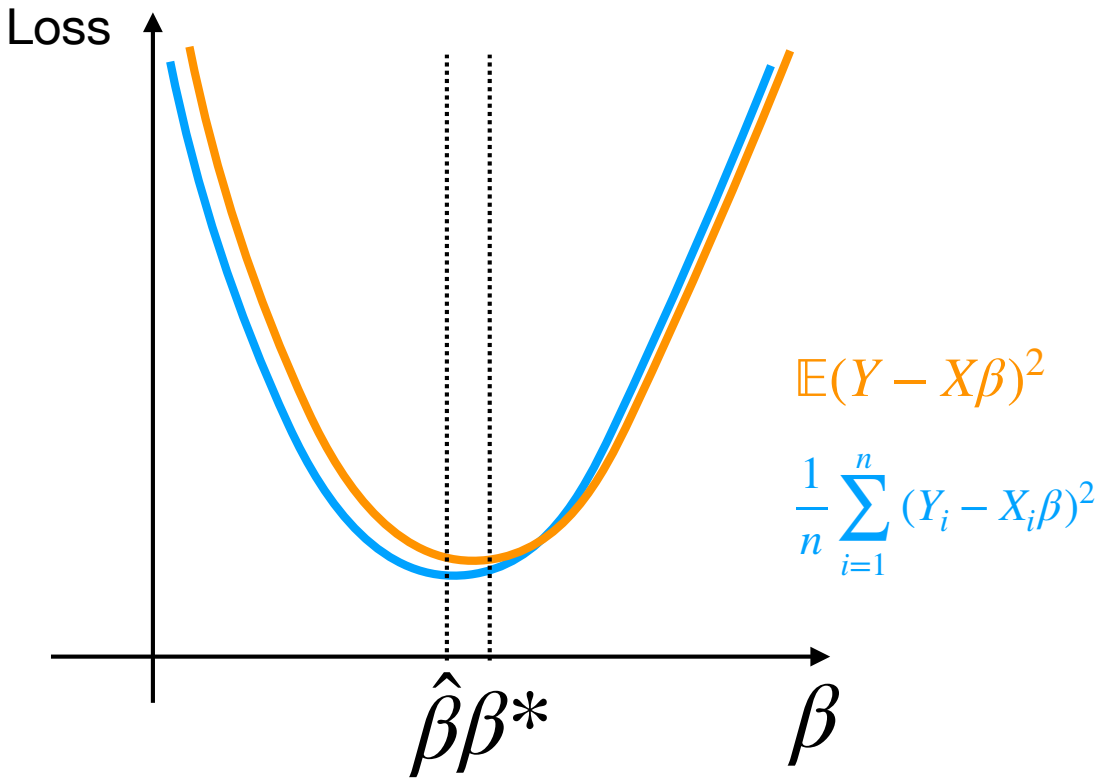
Low-dimensional



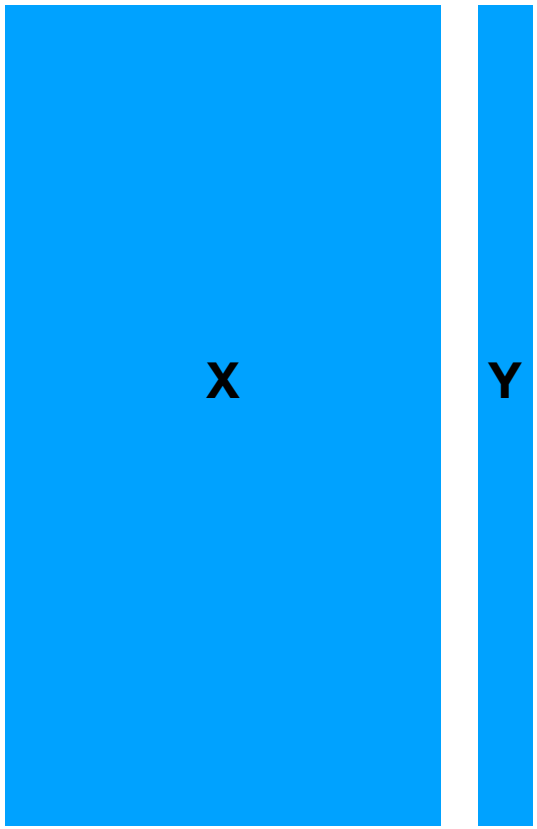
In low dimensional settings…



Low-dimensional

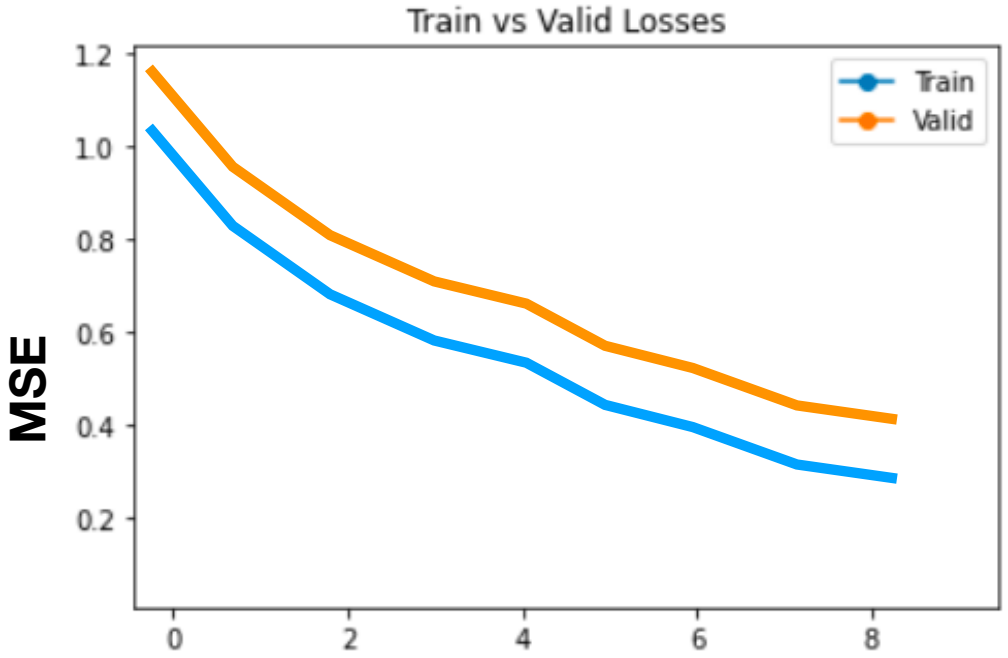


In low dimensional settings…



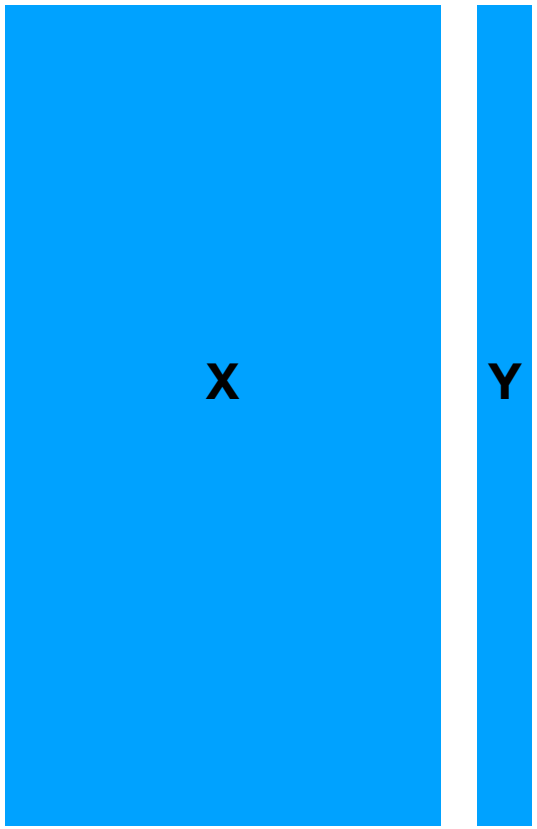
Low-dimensional

Because $\mathbb{E}(Y - X\beta)^2 \approx \frac{1}{n}\sum_{i=1}^{n}(Y_i - X_i\beta)^2$ …



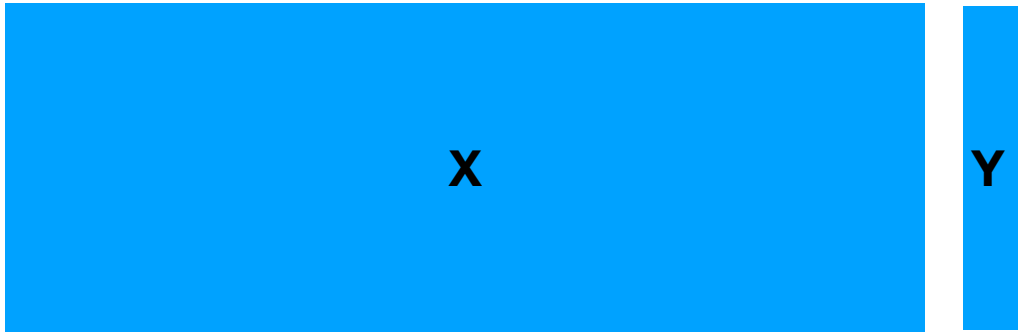
Low vs. High-dimensional data

$n >> p$



Low-dimensional

$p >> n$

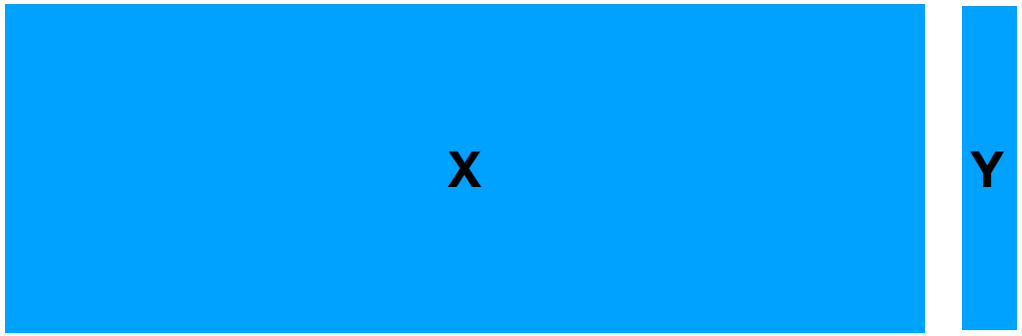


High-dimensional

Examples of high-dimensional settings

- Genome wide association studies
- Electronic health record data
- Deep learning
- …

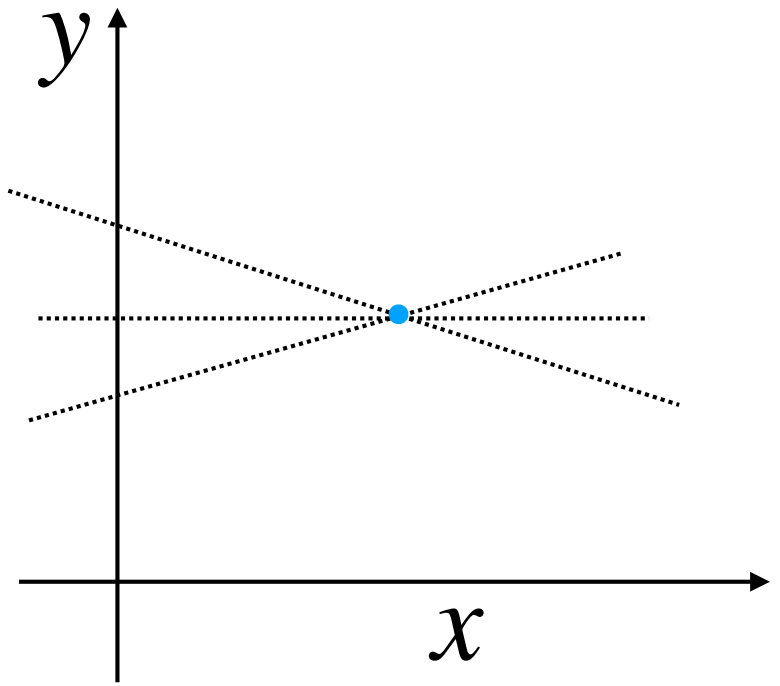
High-dimensional data



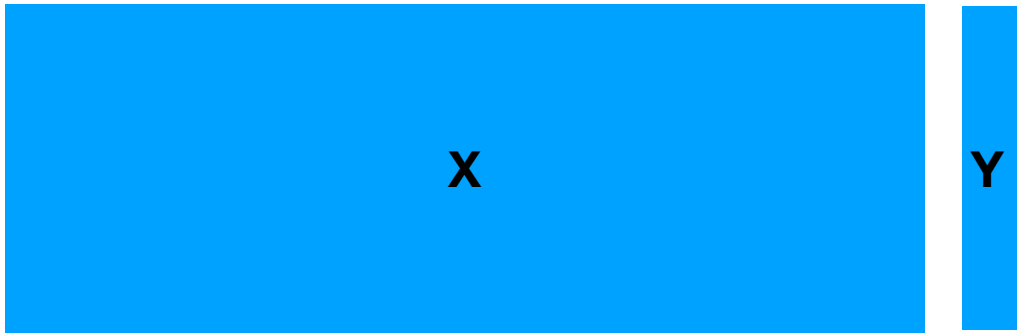
$$p >> n$$

High-dimensional

Example: Suppose we are trying to fit linear regression model by minimizing the MSE. There is only one point, but we have two degrees of freedom: the intercept and coefficient.



High-dimensional data

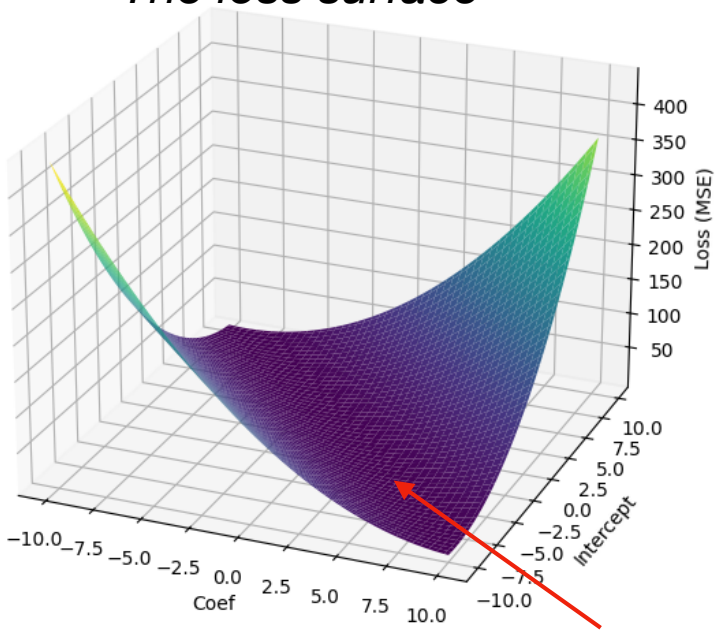


$$p >> n$$

High-dimensional

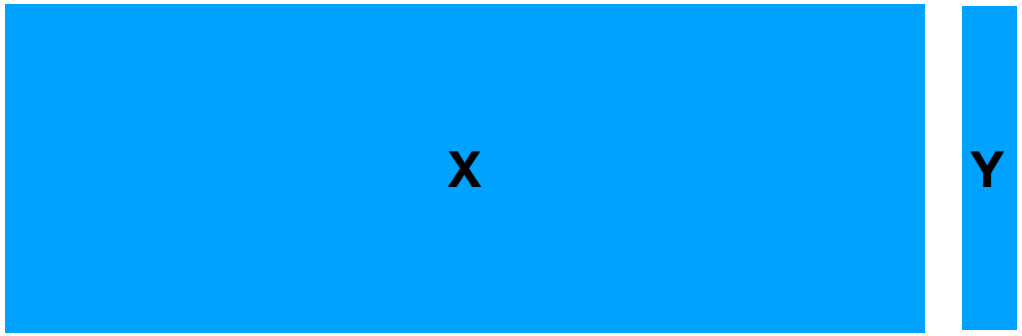
Example: Suppose we are trying to fit linear regression model by minimizing the MSE. There is only one point, but we have two degrees of freedom: the intercept and coefficient.

The loss surface



Notice the valley!

High-dimensional data



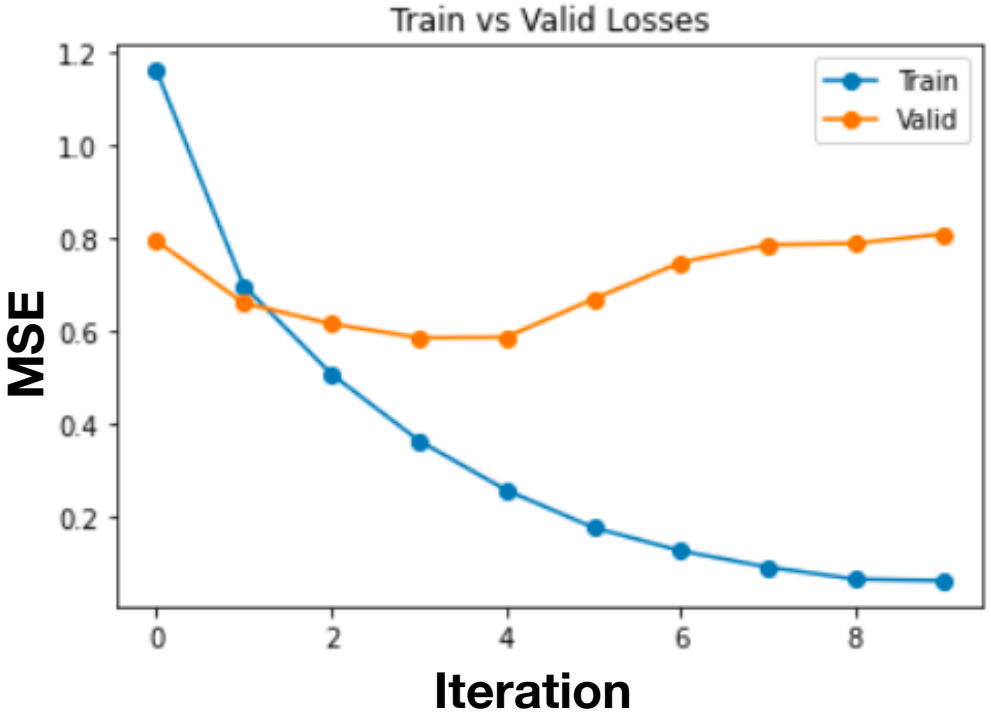
Example: Suppose we are trying to fit linear regression model by minimizing the MSE. There is only one point, but we have two degrees of freedom: the intercept and coefficient.

$$p >> n$$

$$\hat{\beta}_{OLS} = \underbrace{(X^\top X)^{-1}}_{\text{Inverse matrix doesn't exist}} X^\top Y$$

High-dimensional

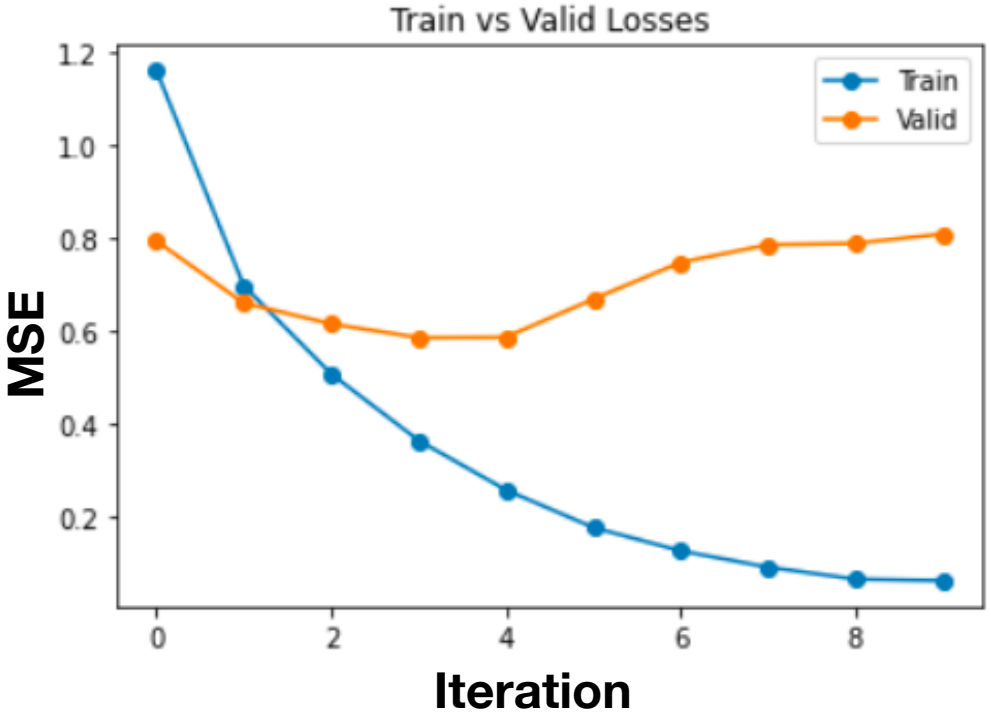
Linear regression in high-dimensional settings



The blue line shows the loss on the training data for the estimate $\hat{\beta}_t$ at iteration t.

The orange line shows the loss on the validation (held-out) data for the estimate $\hat{\beta}_t$ at iteration t.

Linear regression in high-dimensional settings



Q: Why does the orange line go down and then up?

A: Gradient descent failed to minimize the training loss.

B: Minimizing the MSE on the training data can lead to overfitting.

$$\mathbb{E}(Y - X\beta)^2 \quad \approx \quad \frac{1}{n}\sum_{i=1}^{n}(Y_i - X_i\beta)^2$$

Lecture Outline

- Nonlinear regression
- Penalized linear regression
  - **Ridge**
  - Lasso
- Model Complexity

Regularization methods

- Broadly speaking, **regularization** is the process of **adding prior information** to improve the generalizability of the fitted model.

- There are many ways to perform regularization:
  - Penalize high complexity models
  - Bayesian methods that place priors on model parameters
  - Early stopping in deep learning
  - …

$$\min_{\beta} \underbrace{\frac{1}{n}\sum_{i=1}^{n} \ell(Y, f_\beta(X))}_{\text{Loss}} + \lambda \underbrace{J(\beta)}_{\text{Penalty}}$$

Linear regression + regularization

What prior knowledge can we add that can help us better estimate the model?

- Most variables have a small effect on the outcome. Shrink coefficients toward zero. **Ridge regression**

- Most variables have no effect on the outcome. Set most coefficients to zero. **Lasso**

Ridge regression

Recall: Ordinary least squares fits a model by solving the optimization problem

$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2}_{\text{Mean squared error}}$$

Ridge regression fits a model by adding a ridge penalty to the objective function:

$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2}_{\text{Mean squared error}} + \lambda \underbrace{\sum_{j=1}^{p} \beta_j^2}_{\textbf{Ridge penalty}}$$

Ridge regression

Recall: Ordinary least squares fits a model by solving the optimization problem

$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2}_{\text{Mean squared error}}$$

Ridge regression fits a model by adding a ridge penalty to the objective function:

$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2}_{\text{Mean squared error}} + \lambda \underbrace{\|\beta\|_2^2}_{\substack{\textbf{Ridge} \\ \textbf{penalty}}}$$

Fitting ridge regression

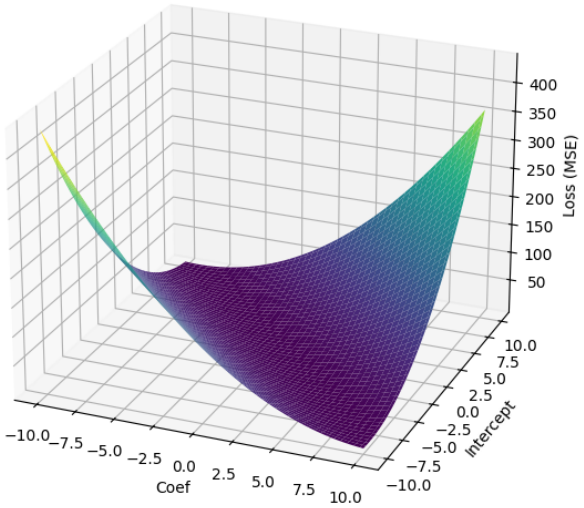
$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2 + \lambda \|\beta\|_2^2$$

- **How can we solve this optimization problem?**
- A: It's not convex. You're doomed.
- B: It's convex, but there are multiple solutions that achieve the global minimum.
- C: It's convex and there is a unique solution. Derive the closed-form solution of the first-order optimality condition.
- D: It's convex and there is a unique solution. Use gradient descent.
- E: It's convex, but none of the above procedures will work…

Fitting ridge regression

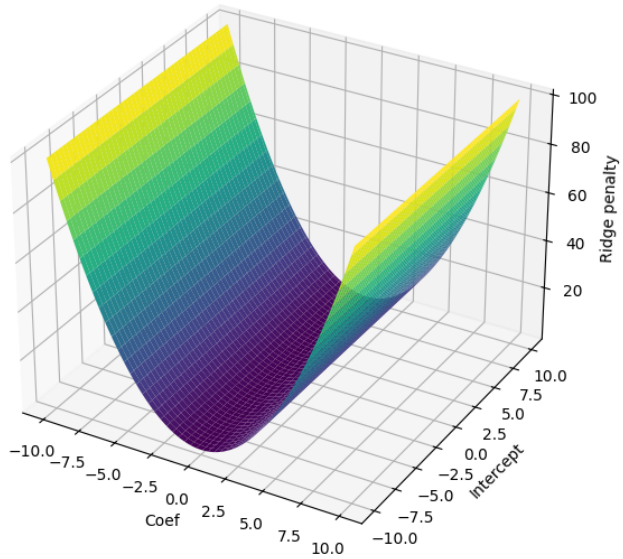
Example: Suppose we are trying to fit linear regression model by minimizing the MSE. There is only one point, but we have two degrees of freedom: the intercept and coefficient.

MSE loss



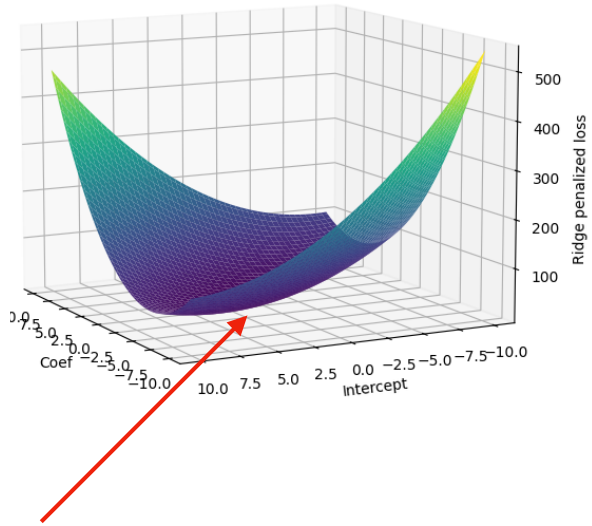
+

Ridge penalty



=

Ridge-penalized loss



Curved at the bottom now.
There is now a unique solution

Fitting ridge regression

$$\min_{\beta \in \mathbb{R}^p} \left\| Y - X\beta \right\|^2 + \lambda \|\beta\|_2^2$$

$$\iff \nabla_\beta \left\{ \left\| Y - X\hat{\beta}_{ridge,\lambda} \right\|^2 + \lambda \|\hat{\beta}_{ridge,\lambda}\|_2^2 \right\} = 0$$

$$\iff -X^\top \left(Y - X\hat{\beta}_{ridge,\lambda} \right) + \lambda \hat{\beta}_{ridge,\lambda} = 0$$

$$\iff (X^\top X + \lambda I)\hat{\beta}_{ridge,\lambda} = X^\top Y$$

$$\iff \hat{\beta}_{ridge,\lambda} = (X^\top X + \lambda I)^{-1} X^\top Y$$

Closed-form solution!

Lecture Outline

- Nonlinear regression
- Penalized linear regression
  - Ridge
  - **Lasso**
- Model Complexity

The Lasso

- **The lasso** is different from ridge regression in that it encourages many model coefficients to be **exactly zero**.

- We say that the resulting model is **sparse** and that the lasso performs **feature selection.**

- The lasso tries to select the optimal combination of features for predicting the response.

The Lasso

Recall: Ordinary least squares fits a model by solving the optimization problem

$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2}_{\text{Mean squared error}}$$

Lasso fits a model by adding a lasso penalty to the objective function:

$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2}_{\text{Mean squared error}} + \lambda \underbrace{\sum_{j=1}^{p} |\beta_j|}_{\textbf{Lasso penalty}}$$

The ridge penalizes β_j^2.

The lasso penalizes $|\beta_j|$.

The Lasso

Recall: Ordinary least squares fits a model by solving the optimization problem

$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2}_{\text{Mean squared error}}$$

Lasso fits a model by adding a lasso penalty to the objective function:

$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \underbrace{\frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2}_{\text{Mean squared error}} + \lambda \underbrace{\|\beta\|_1}_{\substack{\textbf{Lasso} \\ \textbf{penalty}}}$$

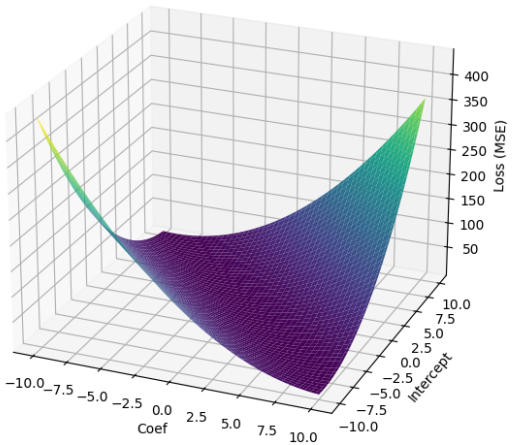
Fitting Lasso regression

$$\min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2 + \lambda \|\beta\|_1$$

- **How can we solve this optimization problem?**
- A: It's not convex. You're doomed.
- B: It's convex, but there are multiple solutions that achieve the global minimum.
- C: It's convex and there is a unique solution. Derive the closed-form solution of the first-order optimality condition.
- D: It's convex and there is a unique solution. Use gradient descent.
- E: It's convex, but none of the above procedures will work…

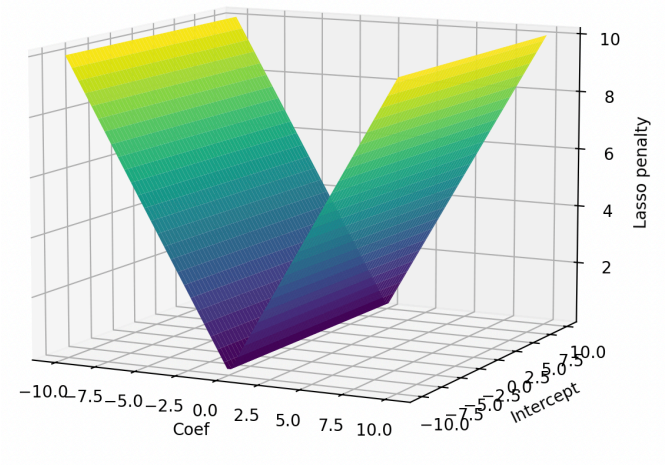
The loss surface

MSE loss



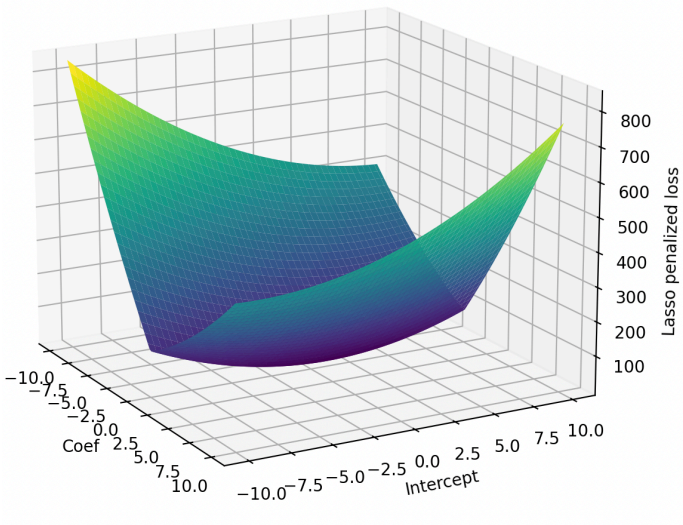
+

Lasso



=

Lasso-penalized loss



Why does the Lasso do feature selection?

That is, why does the Lasso often set some coefficients to exactly zero?

The constrained Lasso

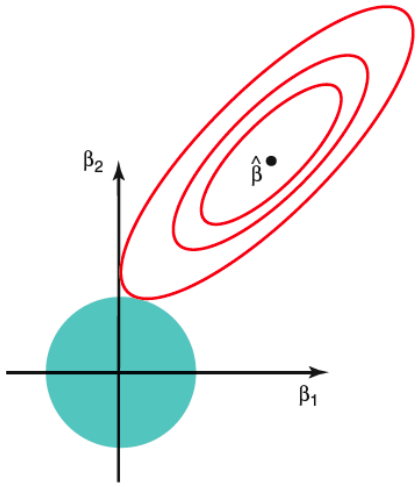
- Ridge Regression is actually equivalent to a constrained optimization problem for some c:

$$\min_{\beta \in \mathbb{R}^p} \left\| Y - X\beta \right\|^2 + \lambda \|\beta\|_2^2 \quad \longleftrightarrow \quad \min_{\beta \in \mathbb{R}^p} \left\| Y - X\beta \right\|^2 \text{ s.t. } \|\beta\|_2^2 \leq c$$

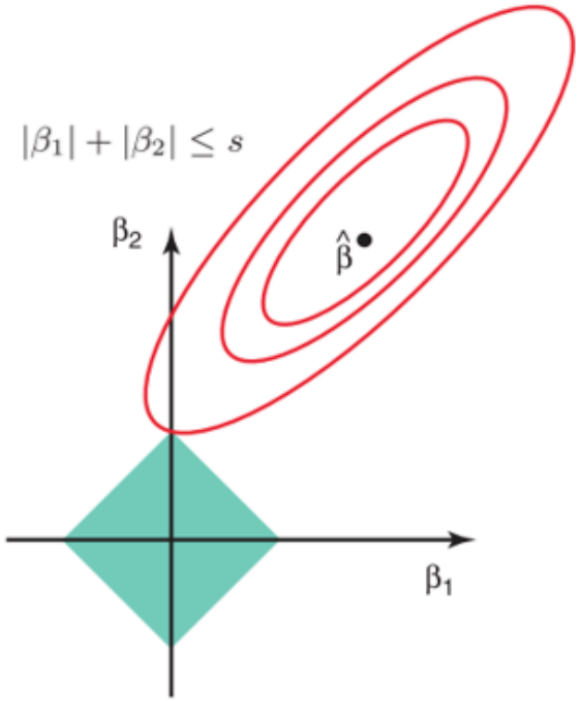
- Similarly, the Lasso is actually equivalent to the constrained lasso problem for some c:

$$\min_{\beta \in \mathbb{R}^p} \left\| Y - X\beta \right\|^2 + \lambda \|\beta\|_1 \quad \longleftrightarrow \quad \min_{\beta \in \mathbb{R}^p} \left\| Y - X\beta \right\|^2 \text{ s.t. } \|\beta\|_1 \leq c$$

The constrained Lasso



*Contours of the error and constraint functions for **ridge** regression. Solid blue area is the constraint region* $\|\beta\|_2^2 < c$ *while the red ellipses are the contours of the mean squared error.*



*Contours of the error and constraint functions for **lasso** regression. Solid blue area is the constraint region* $\|\beta\|_1 < c$ *while the red ellipses are the contours of the mean squared error.*

Why is fitting the lasso tricky

- In all the optimization procedures we've studied up to now, a fundamental assumption is that the objective and constraint functions are ***differentiable everywhere***.

- A function f is ***differentiable*** at x if the derivative exists, i.e. $\lim_{\epsilon \to 0} \frac{f(\beta + \epsilon) - f(\beta)}{\epsilon}$ is well-defined.

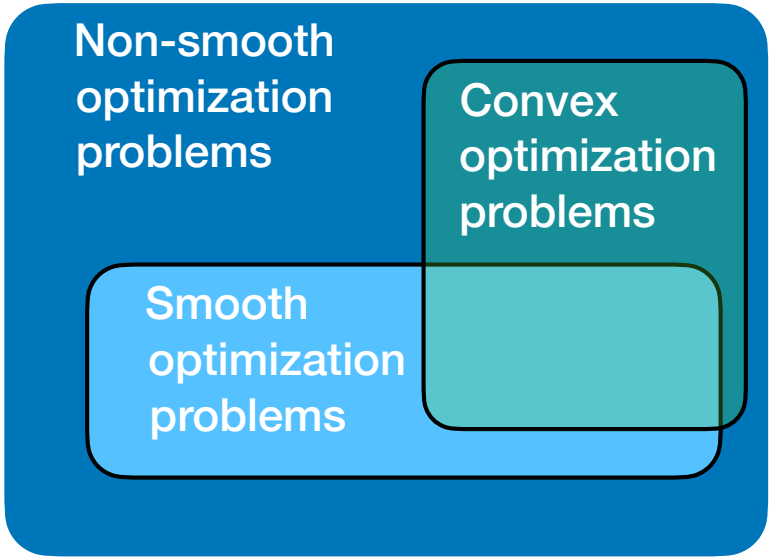
- **Q:** What is the derivative of the lasso penalty $|\beta|$ at $\beta = 0$?

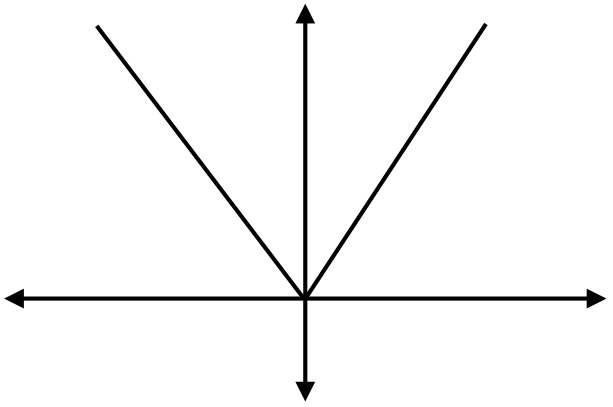
  - Directional derivatives:

    - $\lim_{\epsilon \to 0^+} \frac{|\beta + \epsilon| - |\beta|}{\epsilon} = 1$

    - $\lim_{\epsilon \to 0^-} \frac{|\beta + \epsilon| - |\beta|}{\epsilon} = -1$

  - The derivative doesn't exist!





The lasso is not smooth!

Food for thought

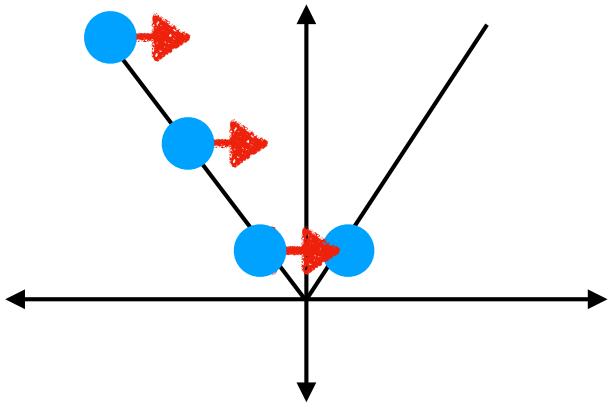
- Q: Can I still run gradient descent to minimize the lasso objective function?
- A: Yes, and you'll reach the global minimum.
- B: Yes, but you might not reach the global minimum.
- C: No, because you can't compute the gradient update step.

Why is fitting the lasso tricky

- The two problems with using gradient descent to solve the lasso are:

  1. GD will never find solutions where the coefficients are exactly zero. Rather, GD will bounce around the zero values.

  2. There is no gradient when some of the coefficients are exactly zero.

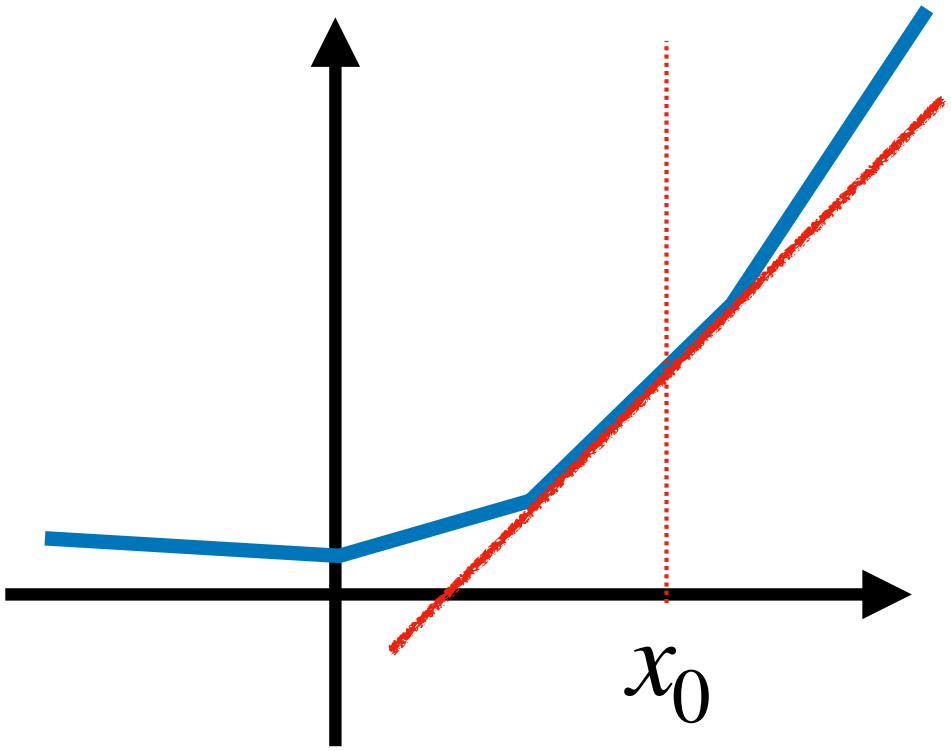


How can we solve the Lasso then?

- There are other optimization procedures specifically designed to solve **non-smooth convex optimization problems**.
  - **Subgradient method**: Replaces the gradient update with a subgradient update.
  - **Proximal gradient descent:** Solve a sequence of "proximal" optimization problems instead, where we approximate the objective locally using a simpler function instead.
  - …

We use a variant of this all the time for training NNs

Subgradient method



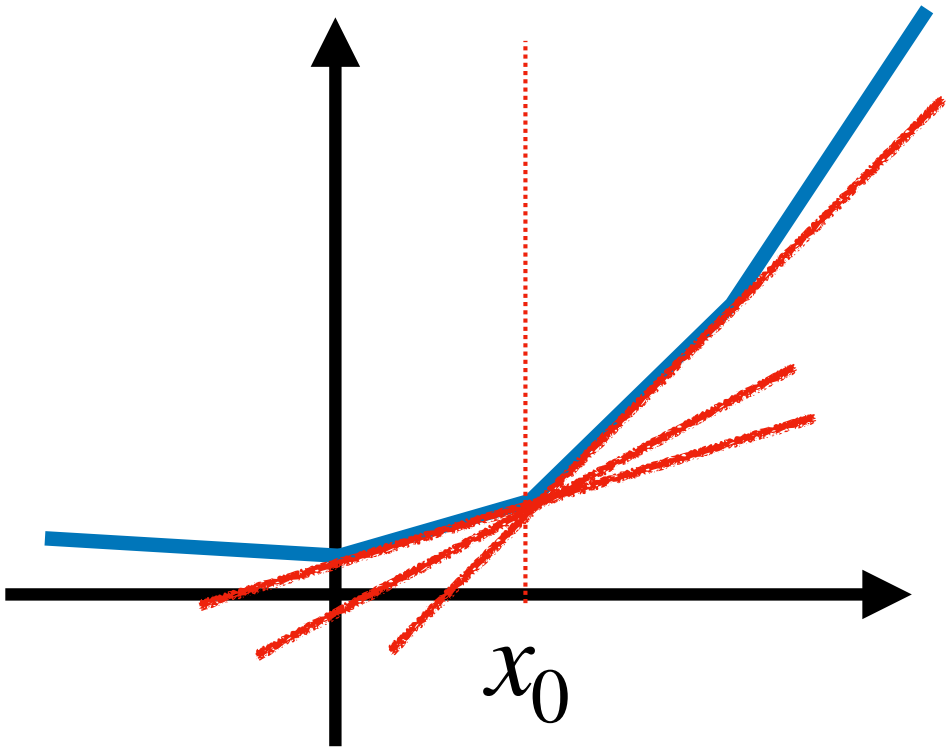
What is the gradient at x_0?

The function is differentiable at x_0:

$$\lim_{x \to x_0^+} \frac{f(x) - f(x_0)}{x - x_0} = \lim_{x \to x_0^-} \frac{f(x) - f(x_0)}{x - x_0} = \lim_{x \to x_0} \frac{f(x) - f(x_0)}{x - x_0}$$

We can also write the gradient at x_0 as the vector v such that $f(x) - f(x_0) = v^\top (x - x_0)$ for all x in a neighborhood of x_0.

Subgradient method



What is the gradient at x_0?

The function is NOT differentiable at x_0:

$$\lim_{x \to x_0^+} \frac{f(x) - f(x_0)}{x - x_0} \neq \lim_{x \to x_0^-} \frac{f(x) - f(x_0)}{x - x_0}$$

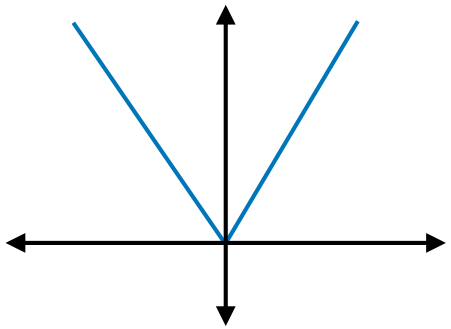
How about subgradients at x_0?

For a real-valued convex function f, the vector v is a subgradient of f at x_0 if $f(x) - f(x_0) \geq v^\top (x - x_0)$.

The set of all subgradients of f at x_0 is denoted $\partial f(x_0)$.

Quiz

- Q: What's the subgradient of the lasso penalty?
- A: {1}
- B: [-1,1]
- C: {-1}
- D: {0}



Subgradient method

Gradient descent

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla f(\theta_{(t-1)})$$

If stopping criterion satisfied:

Return $\theta_{(t)}$

Theorem: For fixed step sizes and a smooth convex optimization problem, gradient descent satisfies

$$\lim_t f(\theta_{(t)}) = \theta^*$$

Subgradient method

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$\theta_{(t)} = \theta_{(t-1)} - \lambda_t g(\theta_{(t-1)})$ where $g(\theta_{(t-1)}) \in \partial f(\theta_{(t-1)})$

If stopping criterion satisfied:

Return $\theta_{(t)}$

Theorem: For diminishing step sizes and a non-smooth Lipschitz convex optimization problem, subgradient method satisfies

$$\lim_t f(\theta_{(t)}) = \theta^*$$

How can we solve the Lasso then?

- There are other optimization procedures specifically designed to solve **non-smooth convex optimization problems**.
  - **Subgradient method**: Replaces the gradient update with a subgradient update.
  - **Proximal gradient descent:** Solve a sequence of "proximal" optimization problems instead, where we approximate the objective locally using a simpler function instead.
  - …

Gradient descent revisited

Gradient Descent

Initialize $\theta_{(0)}$ to some random values.

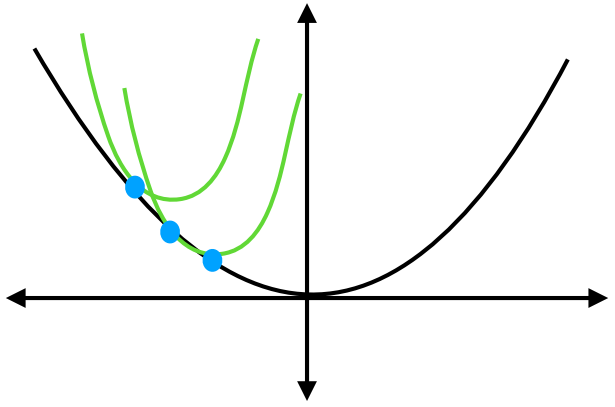
For $t = 1,2,\cdots$

$$\theta_{(t)} = \theta_{(t-1)} - \lambda \nabla f(\theta_{(t-1)})$$

If stopping criterion satisfied:

Return $\theta_{(t)}$

- It turns out that Gradient Descent can be viewed as solving a sequence of ***approximate*** optimization problems instead, where we don't minimize f but local quadratic approximations of f at each iteration:



Gradient descent revisited

Gradient Descent

Initialize $\theta_{(0)}$ to some random values.

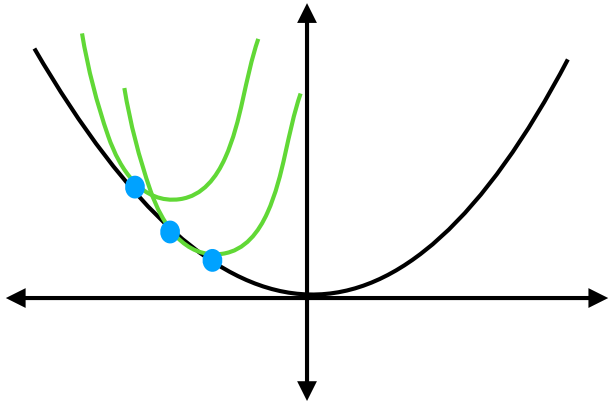
For $t = 1,2,\cdots$

$\theta_{(t)}$ = solution to **approximate** quadratic problem

If stopping criterion satisfied:

Return $\theta_{(t)}$

- It turns out that Gradient Descent can be viewed as solving a sequence of ***approximate*** optimization problems instead, where we don't minimize f but local quadratic approximations of f at each iteration:

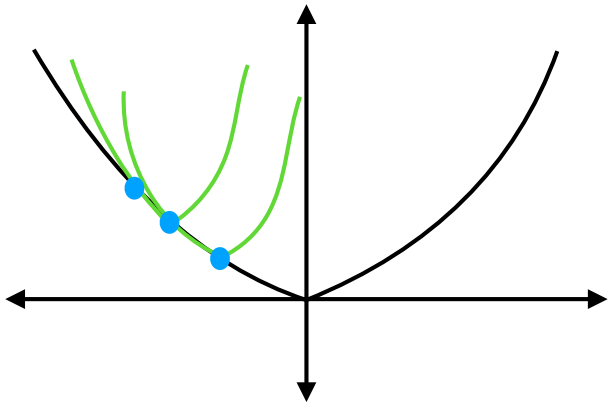


Proximal gradient descent

- Proximal gradient descent generalizes this idea of solving a sequence of approximate problems.

- Suppose our optimization problem has the form $f(x) = g(x) + h(x)$ where $g(x)$ is smooth and $h(x)$ is not smooth.

- At each iteration, the goal is to solve a proximal problem where g is approximated by a quadratic but h is left unchanged.



$$g(x) = \|X\beta - Y\|^2$$
$$h(x) = \lambda\|\beta\|_1$$

Proximal gradient descent

Gradient Descent

$$\min_{\theta} f(\theta)$$

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = \underbrace{\theta_{(t-1)} - \lambda \nabla f(\theta_{(t-1)})}_{\text{solution to approximate quadratic problem}}$$

If stopping criterion satisfied:

Return $\theta_{(t)}$

Proximal Gradient Descent

$$\min_{\theta} g(\theta) + h(\theta)$$

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = \underbrace{prox_h \left\{ \theta_{(t-1)} - \lambda_t \nabla g(\theta_{(t-1)}) \right\}}_{\text{solution to local approximation of } g(\theta) + h(\theta)}$$

If stopping criterion satisfied:

Return $\theta_{(t)}$

Proximal gradient descent for the Lasso

Proximal Gradient Descent

$$\min_{\theta} g(\theta) + h(\theta)$$

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = \underbrace{prox_h \left\{ \theta_{(t-1)} - \lambda_t \nabla g(\theta_{(t-1)}) \right\}}_{\substack{\textit{solution to local approximation} \\ \textit{of } g(\theta) + h(\theta)}}$$

If stopping criterion satisfied:

Return $\theta_{(t)}$

Iterative soft-thresholding algorithm (ISTA)

$$\min_{\theta} \|Y - X\beta\|^2 + \lambda \|\theta\|_1$$

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = S_{\lambda t} \left(\theta_{(t-1)} - \gamma \nabla \left(\|Y - X\theta_{(t-1)}\|^2 \right) \right)$$

If stopping criterion satisfied:

Return $\theta_{(t)}$

Proximal gradient descent for the Lasso

Iterative soft-thresholding algorithm (ISTA)

$$\min_{\theta} \|Y - X\beta\|^2 + \lambda\|\theta\|_1$$

Initialize $\theta_{(0)}$ to some random values.

For $t = 1,2,\cdots$

$$\theta_{(t)} = S_{\lambda t}\left(\theta_{(t-1)} - \gamma\nabla\left(\|Y - X\theta_{(t-1)}\|^2\right)\right)$$

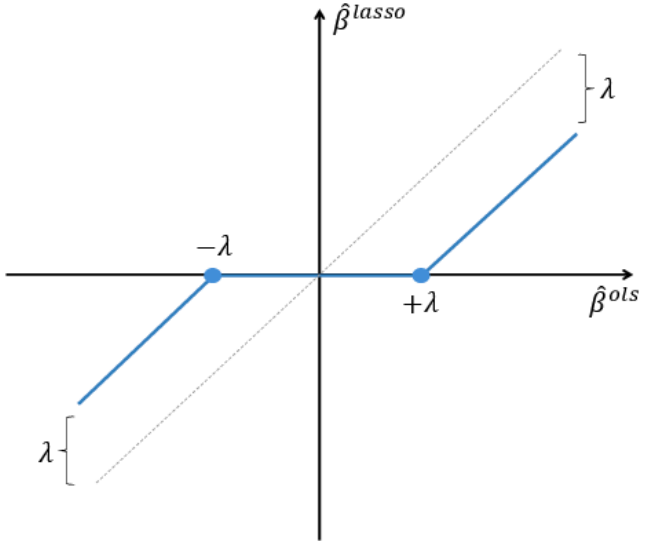
If stopping criterion satisfied:

Return $\theta_{(t)}$

1. Take a gradient step.

2. Soft-thresholds coefficients towards zero.

Soft-threshold function $S_{\lambda t}(\beta)$



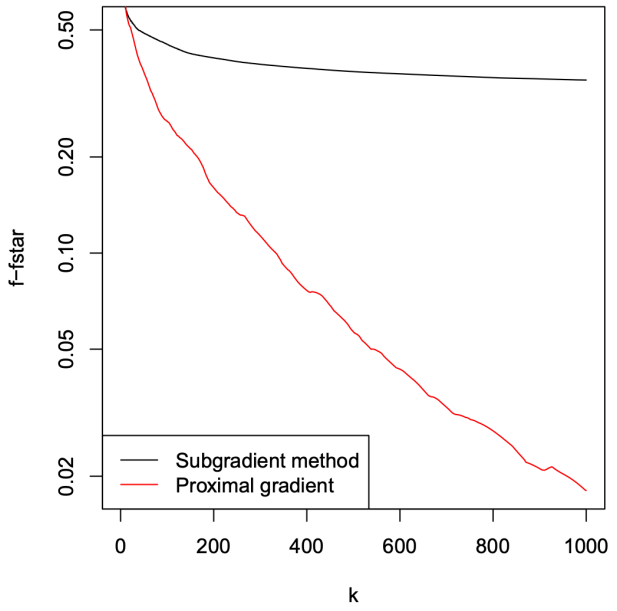
How can we solve the Lasso then?

- There are other optimization procedures specifically designed to solve **non-smooth convex optimization problems**.
  - **Subgradient method**: Replaces the gradient update with a subgradient update.
  - **Proximal gradient descent:** Solve a sequence of "proximal" optimization problems instead, where we approximate the objective locally using a simpler function instead.
  - …

How can we solve the Lasso then?

- There are other optimization procedures specifically designed to solve **non-smooth convex optimization problems**.
  - **Subgradient method**: Replaces the gradient update with a subgradient update.
  - **Proximal gradient descent:** Solve a sequence of "proximal" optimization problems instead, where we approximate the objective locally using a simpler function instead.
  - …

Which one should we choose?

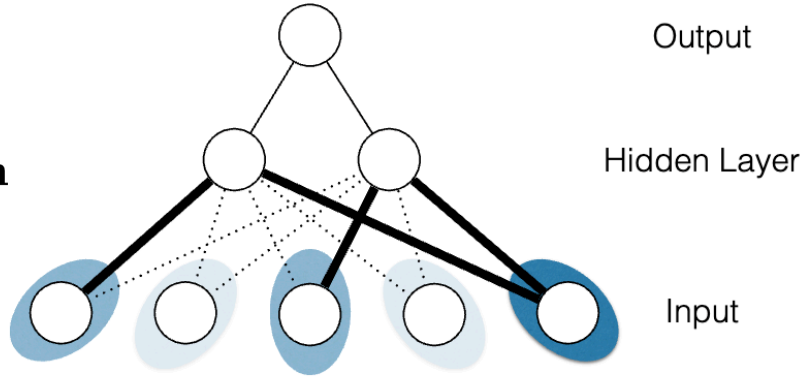


Fancier penalization methods

- **Group lasso** $\|\beta\|_2$: Penalizes groups of coefficients to be zero together
- **Nuclear norm penalization** $\|M\|_*$**:** Penalizes the rank of a matrix (of coefficients)
- **Mixtures of penalties:**
  - **Elastic Net:** Lasso + Ridge
  - **Sparse group lasso:** Group lasso + Lasso
  - …

Fancier models + penalization

Sparse-Input Neural Networks for High-dimensional Nonparametric Regression and Classification



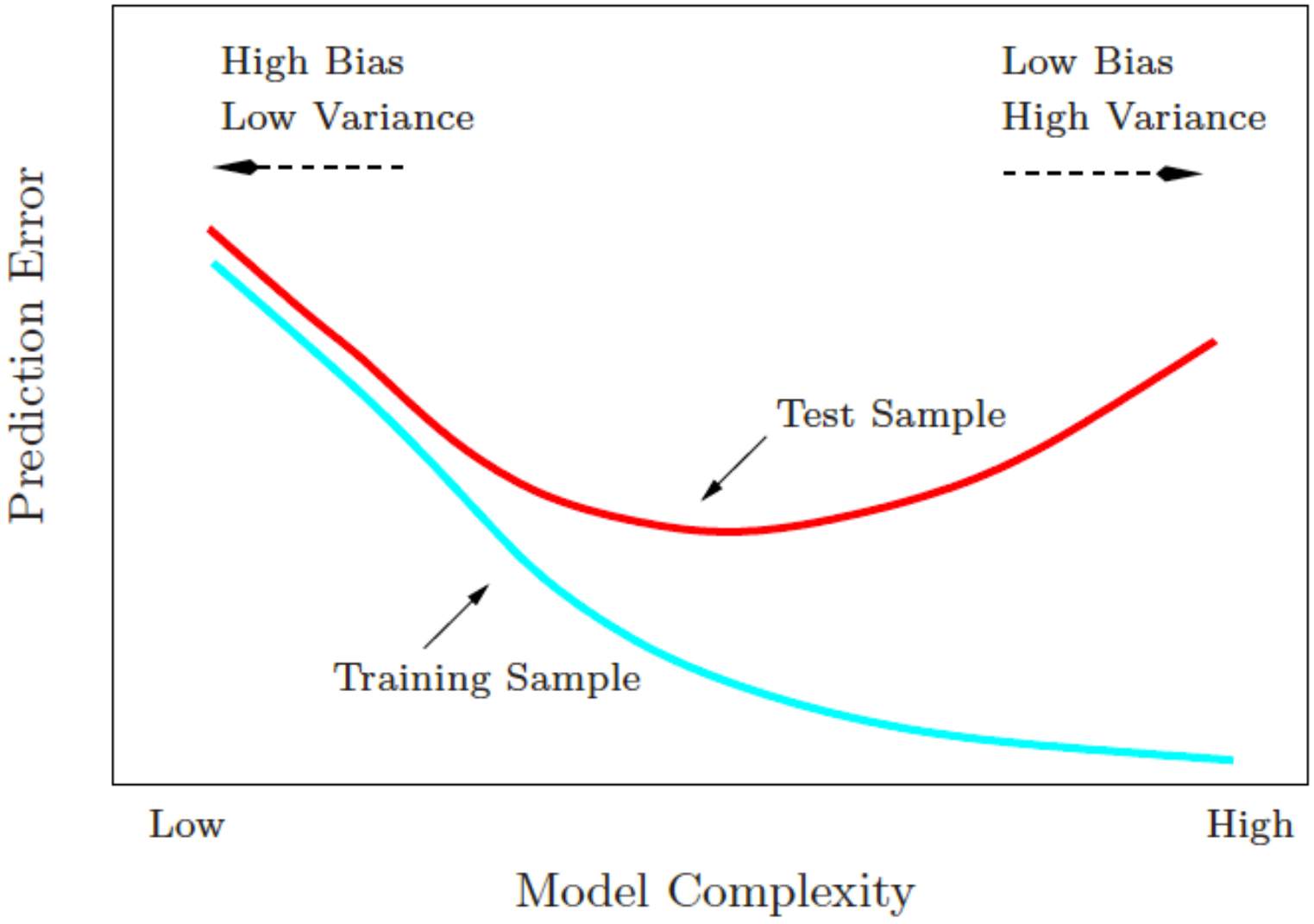
$$\arg\min_{\eta} \frac{1}{n} \sum_{i=1}^{n} \ell\left(y_i, f_{\boldsymbol{\eta}}(\boldsymbol{x}_i)\right) + \lambda_0 \sum_{a=2}^{L+1} \|\boldsymbol{\theta}_a\|_2^2 + \lambda \sum_{j=1}^{p} \Omega_\alpha(\boldsymbol{\theta}_{1,\cdot,j})$$

where $\Omega_\alpha(\boldsymbol{\theta}) = (1-\alpha)\|\boldsymbol{\theta}\|_1 + \alpha\|\boldsymbol{\theta}\|_2$ is the sparse group lasso

Lecture Outline

- Nonlinear regression
- Penalized linear regression
  - Ridge
  - Lasso
- **Model Complexity**

Bias-Variance Tradeoff



Quiz

- Q: Consider Ridge regression with penalty parameter λ.
- Ridge: $\hat{\beta}_\lambda = \min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0) \right)^2 + \lambda \|\beta\|_2^2$

When λ decreases towards 0, are we more or less likely to overfit to the data?

- A: Ridge will be more likely to overfit (higher model complexity)
- B: Ridge will be less likely to overfit (lower model complexity)

Quiz

- Q: Consider the Lasso with penalty parameter λ.
- Lasso: $\hat{\beta}_\lambda = \min_{\beta \in \mathbb{R}^p, \beta_0 \in \mathbb{R}} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (x_i^\top \beta + \beta_0)\right)^2 + \lambda \|\beta\|_1$

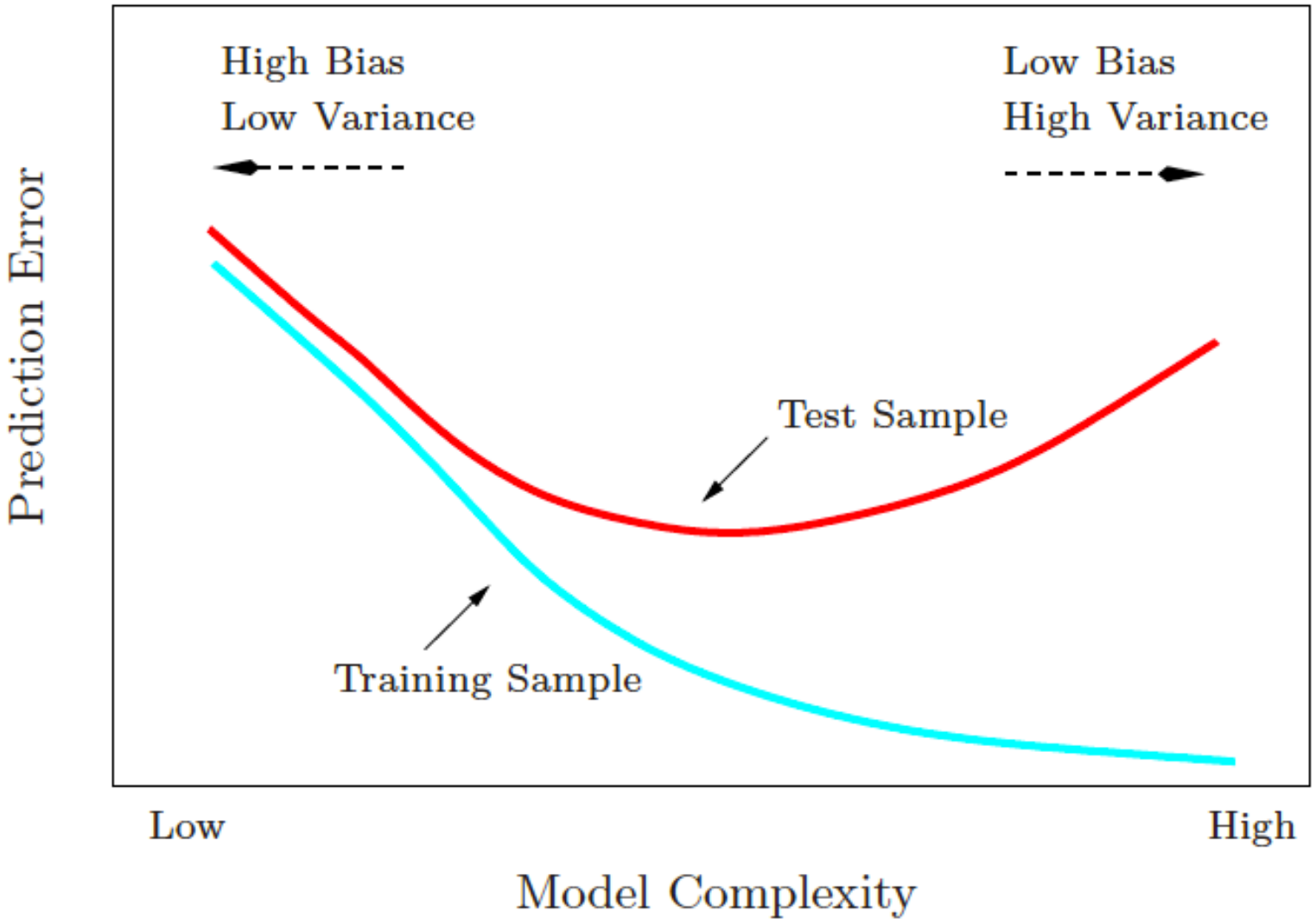
When λ decreases towards 0, are we more or less likely to overfit to the data?

- A: Lasso will be more likely to overfit (higher model complexity)
- B: Lasso will be less likely to overfit (lower model complexity)

Quiz

- **Q: How should we pick λ?**
- **A:** We should pre-specify the value of λ.
- **B:** Pick the λ that minimizes the training error.
- **C:** Pick the λ that minimizes the test error.
- **D:** We use sample-splitting or cross-validation.

Bias-Variance Tradeoff



But what is model complexity exactly?

Model generalization

- Generalization error (or Prediction Error): $\mathbb{E}[(Y - \hat{f}(X))^2]$
- Suppose $Y = f(X) + \epsilon$ where $\mathbb{E}[\epsilon] = 0$.
- Bias-variance decomposition:

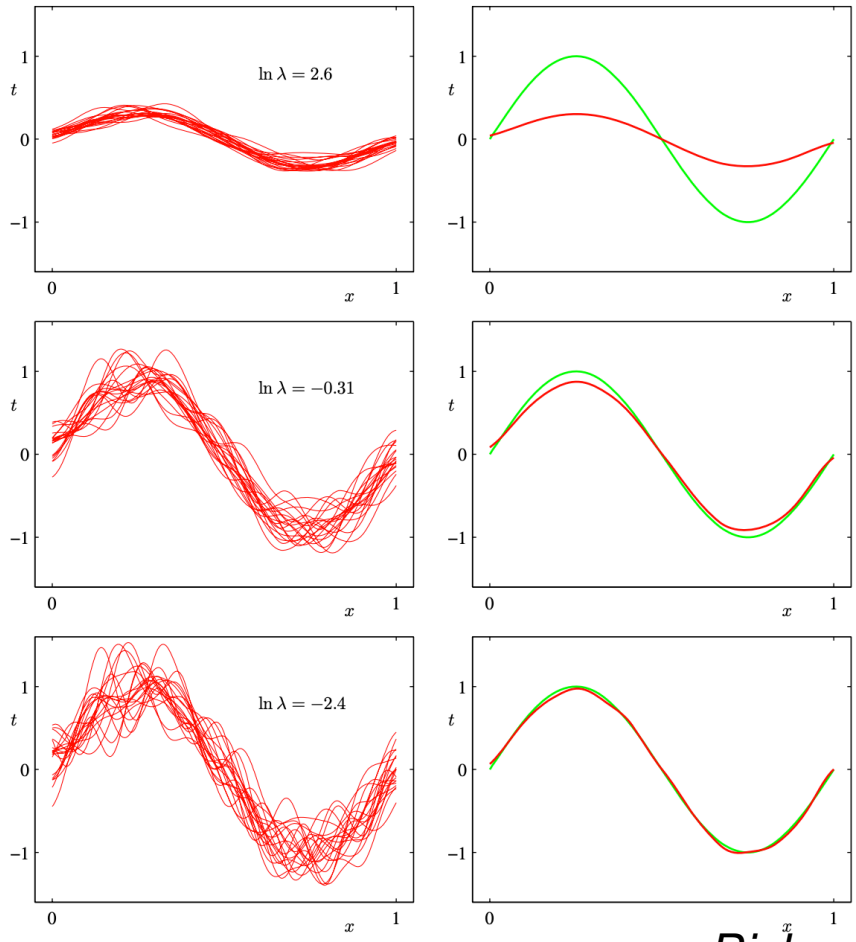
$$\begin{aligned}\mathbb{E}[(Y - \hat{f}(X))^2] &= \mathbb{E}[(\epsilon + f(X) - \hat{f}(X))^2] \\ &= \underbrace{\mathbb{E}[\epsilon^2]}_{\substack{\textit{irreducible} \\ \textit{error}}} + \underbrace{\mathbb{E}[f(X) - \hat{f}(X)]^2}_{Bias^2} + \underbrace{\mathbb{E}[(f(X) - \hat{f}(X))^2] - \mathbb{E}[f(X) - \hat{f}(X)]^2}_{=Var(f(X) - \hat{f}(X))}\end{aligned}$$

Model generalization

$$\mathbb{E}[(Y-\hat{f}(X))^2] = \underbrace{\mathbb{E}[\epsilon^2]}_{\text{irreducible error}} + \underbrace{\mathbb{E}[f(X)-\hat{f}(X)]^2}_{Bias^2} + \underbrace{\mathbb{E}[(f(X)-\hat{f}(X))^2] - \mathbb{E}[f(X)-\hat{f}(X)]^2}_{=Var(f(X)-\hat{f}(X))}$$

Consider a model class where we fit a linear model but with 24 Gaussian basis functions. We have a penalty parameter λ, which control how large the coefficients are in the model and thus control how squiggly the model is.

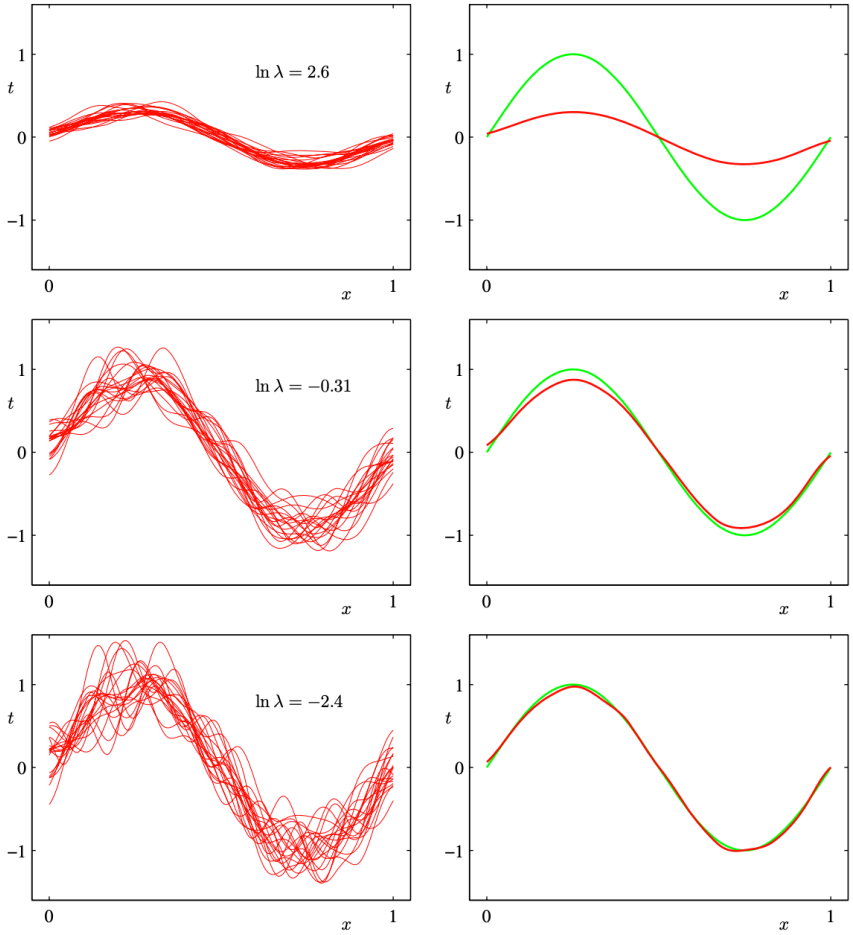
$$\min_{\beta\in\mathbb{R}^p,\beta_0\in\mathbb{R}} \frac{1}{n}\sum_{i=1}^{n}\left(y_i-(x_i^\top\beta+\beta_0)\right)^2+\lambda\|\beta\|_2^2$$



Bishop Ch 3

Model complexity

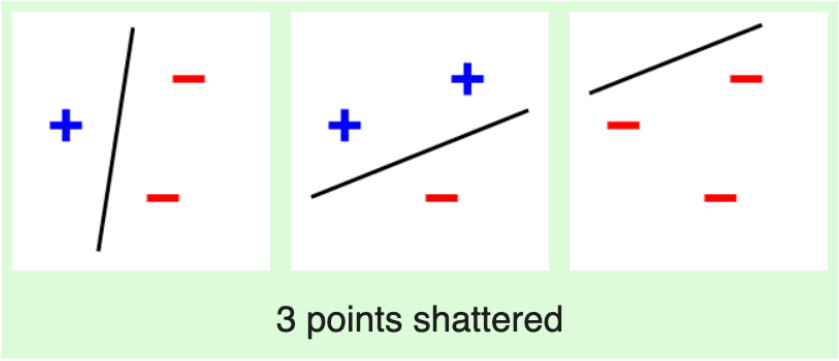
- Given an estimation procedure f_λ, its **model complexity/ capacity** quantifies how many different functions it can end up learning.



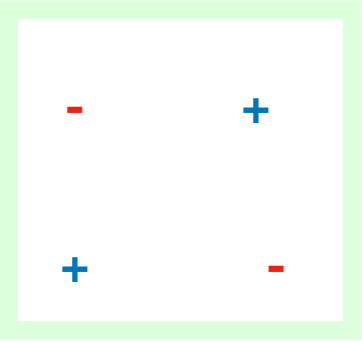
Model complexity

- There are many measures of model complexity. In classification, the **Vapnik–Chervonenkis (VC) dimension** is one of the most widely used measures.
  - Suppose we have a dataset of n points. A model class H is said to "**shatter**" the dataset if for any labeling of the n points, there is an $h \in H$ such that $h(X_i) = Y_i$ for all $i = 1, \cdots, n$.
  - The VC dimension of a model class H is the largest n such that there exists a generally positioned dataset of size n that can be shattered by H.

In a 2D space, a line can shatter 3 points:



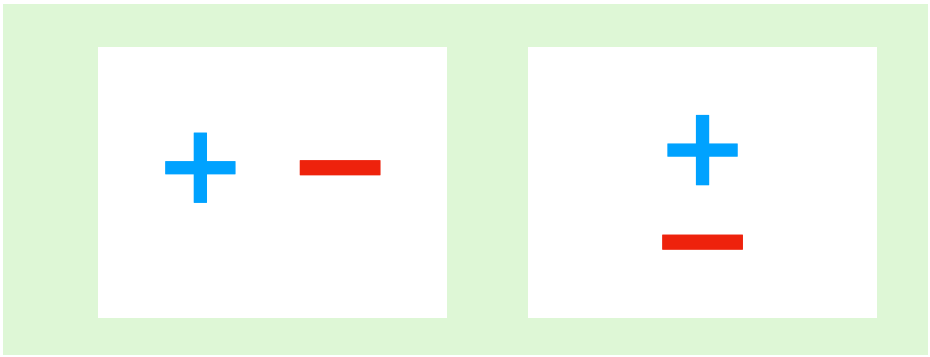
But can a line shatter 4 points?



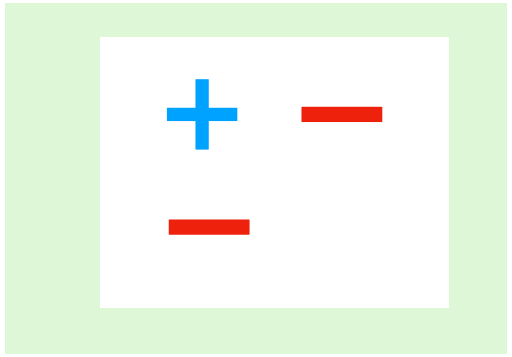
Model complexity

- How does the lasso control the VC dimension?
  - Suppose we chose a lasso penalty such that the learned model will only have one coefficient that is non-zero.
  - How does that affect the ability of the model class to shatter points?

Datasets with 2 points

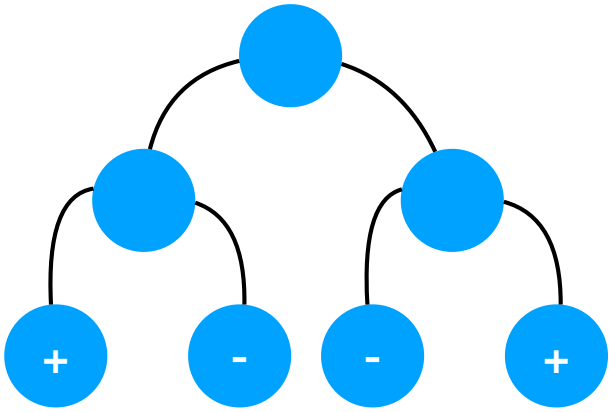


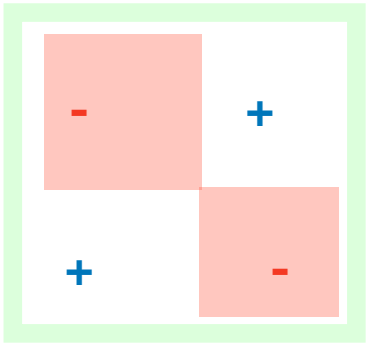
A dataset with 3 points

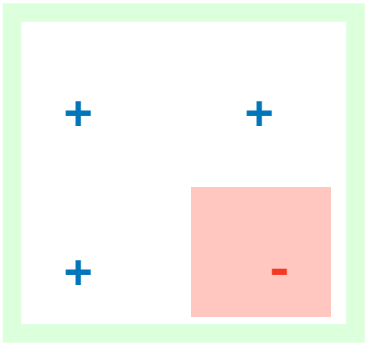


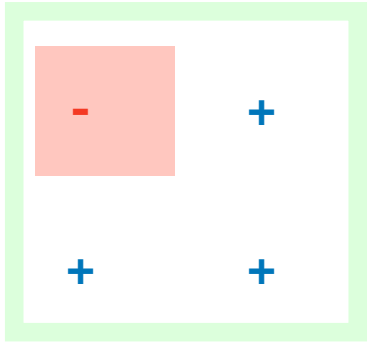
Model complexity

So a line cannot shatter 4 points in 2D settings… What is a model class that can?









Intuitively, why does the VC dimension help us characterize generalization error?

A model class that can shatter more points can memorize larger datasets.

The more a model can memorize, the more it tends to overfit to the training data.

Model complexity

- One can bound the generalization error of a ML model in terms of its VC dimension!

- If a ML model has VC dimension D and there are N training observations, then the test error will not be significantly larger than the training error with high probability per the following:

$$\Pr\left(\text{test error} \leqslant \text{training error} + \sqrt{\frac{1}{N}\left[D\left(\log\left(\frac{2N}{D}\right)+1\right)-\log\left(\frac{\eta}{4}\right)\right]}\right) = 1-\eta,$$

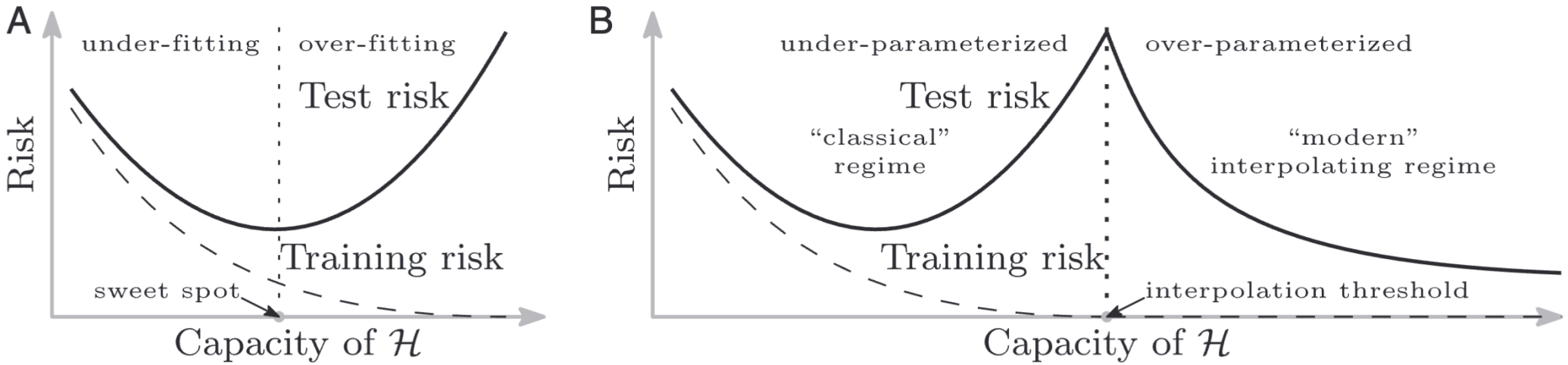
"Oracle inequality"

Lecture Outline

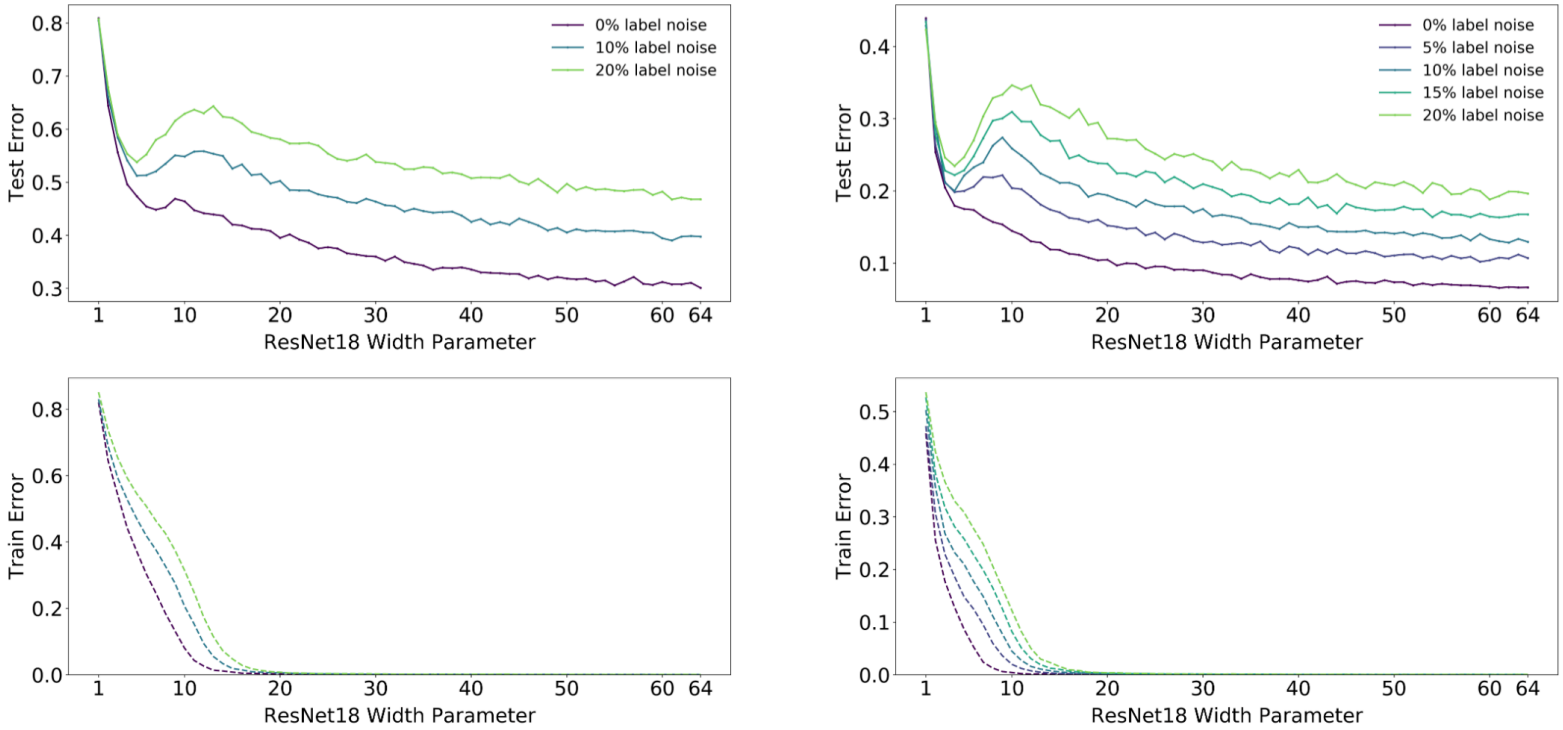
- Nonlinear regression
- Penalized linear regression
  - Ridge
  - Lasso
- Model Complexity

Double-descent

- The classical understanding in machine learning is that there is a bias-variance tradeoff and the typical visualization is a U-shaped curve (Fig A).
- **However, modern neural networks exhibit no such phenomenon.** Such networks have millions of parameters, more than enough to fit even random labels (Zhang et al. (2016)), and yet they perform much better on many tasks than smaller models. Indeed, conventional wisdom among practitioners is that “larger models are better” Krizhevsky et al. (2012), Huang et al. (2018)).
- **New research has identified a new phenomenon named “Double-Descent”** (Belkin et. al., PNAS 2019, Fig B): Increasing model capacity beyond the point of interpolation can result in improved performance.



Double-descent



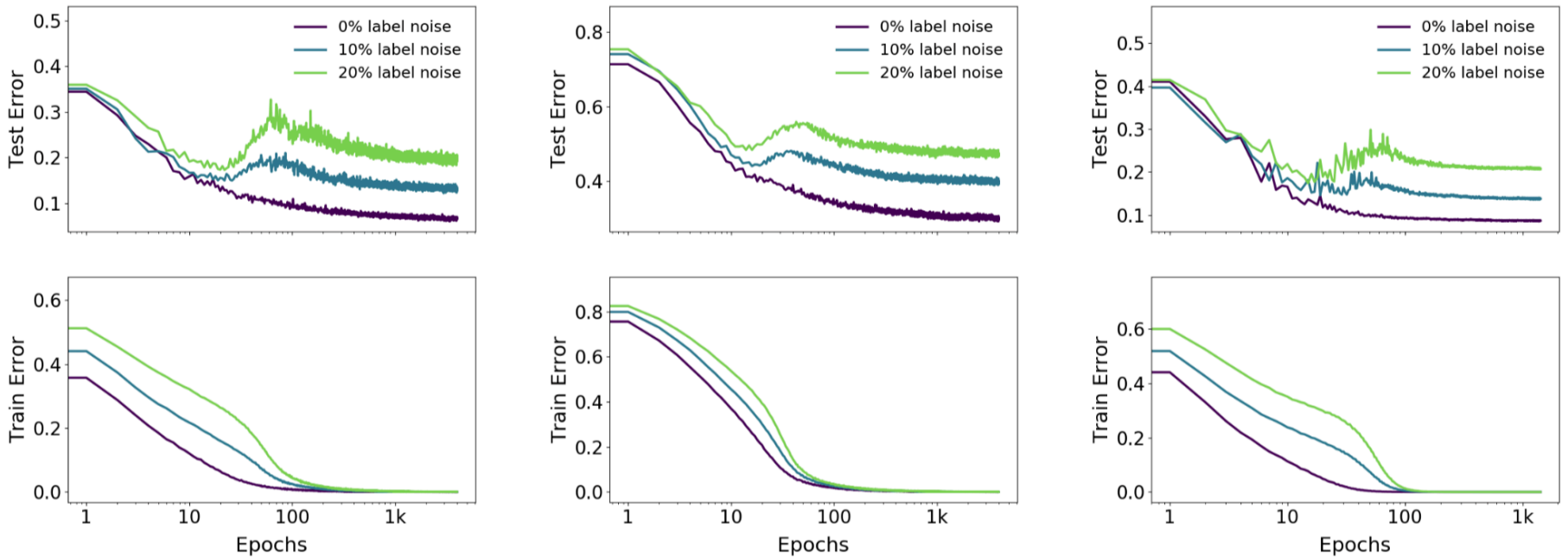
(a) **CIFAR-100.** There is a peak in test error even with no label noise.

(b) **CIFAR-10.** There is a "plateau" in test error around the interpolation point with no label noise, which develops into a peak for added label noise.

Figure 4: **Model-wise double descent for ResNet18s.** Trained on CIFAR-100 and CIFAR-10, with varying label noise. Optimized using Adam with LR 0.0001 for 4K epochs, and data-augmentation.

- Nakkarim et. al., ICLR 2020

Double-descent



(a) ResNet18 on CIFAR10. (b) ResNet18 on CIFAR100. (c) 5-layer CNN on CIFAR 10.

Figure 10: **Epoch-wise double descent** for ResNet18 and CNN (width=128). ResNets trained using Adam with learning rate 0.0001, and CNNs trained with SGD with inverse-squareroot learning rate.

Double-descent

- **Belkin et. Al. PNAS 2019:** "The answer is that the capacity of the function class does not necessarily reflect how well the predictor matches the inductive bias appropriate for the problem at hand. For the learning problems we consider (a range of real-world datasets as well as synthetic data), the inductive bias that seems appropriate is the regularity or smoothness of a function as measured by a certain function space norm. Choosing the smoothest function that perfectly fits observed data is a form of Occam's razor: The simplest explanation compatible with the observations should be preferred… By considering larger function classes, which contain more candidate predictors compatible with the data, we are able to find interpolating functions that have smaller norm and are thus "simpler." Thus, increasing function class capacity improves performance of classifiers.

- **An analog of model-wise double descent occurs even for linear models** (see e.g. Hastie et al. [2019]): The idea is that in higher dimensions, one might able to find a "smoother" (smaller L2 norm) model that explains the data well, whereas in the constrained low-dim space one might not be able to explain the data well. This is particularly true when the model is misspecified. In this higher-dimensional space, the learned model has lower variance if we are always trying to find the model with zero MSE but also smallest L2 norm AND it can have lower bias because we are expanding the model space.

- Still an active research topic…